ARRAY

ONE-DIMENSIONAL

prepared by:

Gyro A. Madrona

Electronics Engineer

GRADING SYSTEM

FINAL GRADE

30%

PRELIM

30%

MIDTERM

40%

ENDTERM

TERM GRADE

50%

WEEKLY

50%

EXAM

100%

LABORATORY

PENALTY

0

ABSENT

3

45-min LATE

7 max

15-min LATE

TOPIC OUTLINE

One-dimensional Array

- Declaration
- Initialization
- Accessing values

ONE-DIMENTIONAL ARRAY

An array is a collection of **same-type** elements stored contiguously in memory, accessible by **indexing**

	0	1	2	3	4
num	7	5	1	3	9

An array is a collection of same-type elements stored contiguously in memory, accessible by indexing

	0	1	2	3	4
num	7	5	1	3	9

name

An array is a collection of same-type elements stored contiguously in memory, accessible by indexing

index

	0	1	2	3	4
num	7	5	1	3	9

An array is a collection of same-type elements stored contiguously in memory, accessible by indexing

num

0	1	2	3	4
7	5	1	3	9

values

DECLARATION

```
int num_1;
int num_2;
int num_3;
int num_4;
int num_5;
```

Syntax:

data type name [elements] ;

int num [5] ;

index starts at 0

	0	1	2	3	4
num	?	?	?	?	?

DECLARATION + INITIALIZATION

```
int num_1 = 7;
int num_2 = 5;
int num_3 = 1;
int num_4 = 3;
int num_5 = 9;
```

Syntax:

data type name [elements] = { } ;

int num [5] = { 7, 5, 1, 3, 9 } ;

int num [ ] = { 7, 5, 1, 3, 9 } ;

	0	1	2	3	4
num	7	5	1	3	9

ACCESING VALUES

Syntax:

name [index]

To access the value 5,

value = **num[1]**;

To access the value 9,

value = **num[4]**;

	0	1	2	3	4
num	7	5	1	3	9

ACCESING VALUES

Syntax:

name [index]

```
int num[ ] = { 7, 5, 1, 3, 9 };

for(int i = 0; i <= 4; i++) {
    cout<< num[ i ] <<endl;
}
```

	0	1	2	3	4
num	7	5	1	3	9

LABORATORY